

Chapitre 4

Data Visualisation

Prof. Essaid EL BACHARI

Laboratoire Ingénierie des Systèmes Informatiques

Département d'Informatique - Faculté des Sciences Semlalia

Université Cadi Ayyad Marrakech

Introduction

Objectif : Nous allons explorer les principes essentiels pour créer des visualisations de données efficaces et informatives.

Définition : La visualisation de données transforme des chiffres et des statistiques en récits visuels, facilitant ainsi la compréhension, l'analyse et la prise de décision.

Principes

- **Clarté et Simplicité** : Comment transmettre l'information sans surcharge visuelle.
- **Précision et Intégrité** : Assurer que les visualisations reflètent fidèlement les données.
- **Pertinence et Contexte** : Adapter chaque graphique au message et à l'audience.
- **Cohérence** : Maintenir un style et une utilisation des couleurs uniformes.

Types de graphiques

Les 5 visualisations les plus utilisées en data science sont :

- ❖ **Histogramme** : Montre la distribution d'une variable continue (ex : la répartition des âges, revenus, etc.).
- ❖ **Nuage de points (Scatter Plot)** : Explore la relation entre deux variables numériques (ex : taille vs poids).
- ❖ **Heatmap (Carte de chaleur)** : Visualise la corrélation entre plusieurs variables numériques
- ❖ **Boîte à moustaches (Box Plot)** : Affiche la distribution d'une variable et détecte les valeurs aberrantes.
- ❖ **Diagramme en barres (Bar Plot)** : Compare des catégories (ex : ventes par produit ou par région).

Les diagramme en rouge seront traités dans ce cours. Les 2 autres dans les TPs.

Bibliothèques de visualisation

Les bibliothèques de visualisation les plus populaires en data science avec Python sont : (en rouge biblio qui seront

- **Matplotlib** : La base pour les graphiques 2D en Python.
- **Seaborn** : Extension de Matplotlib pour des visualisations statistiques améliorées.
- **Bokeh** : Visualisations interactives pour le web, adaptée aux grands ensembles de données.
- **Plotly** : *Graphiques interactifs et esthétiques, idéaux pour le web.*
- **Altair** : *Visualisations déclaratives, expressives et simples.*

Matplotlib

- Matplotlib est une bibliothèque Python incontournable pour la création de graphiques. Elle permet de créer des visualisations simples mais puissantes.
- Apprendre à utiliser les graphiques de base permettant d'explorer et d'analyser les données tels que :
 - **Le scatter plot,**
 - **Le box plot,**
 - **L'histogramme.**

Matplotlib - Scatter Plot

- **Un scatter plot** est utilisé pour visualiser la relation entre deux variables numériques continues.
- **Utilisation** : Idéal pour observer des corrélations entre deux variables.
- **Exemple** : *Relation entre la taille et le poids des individus ?*

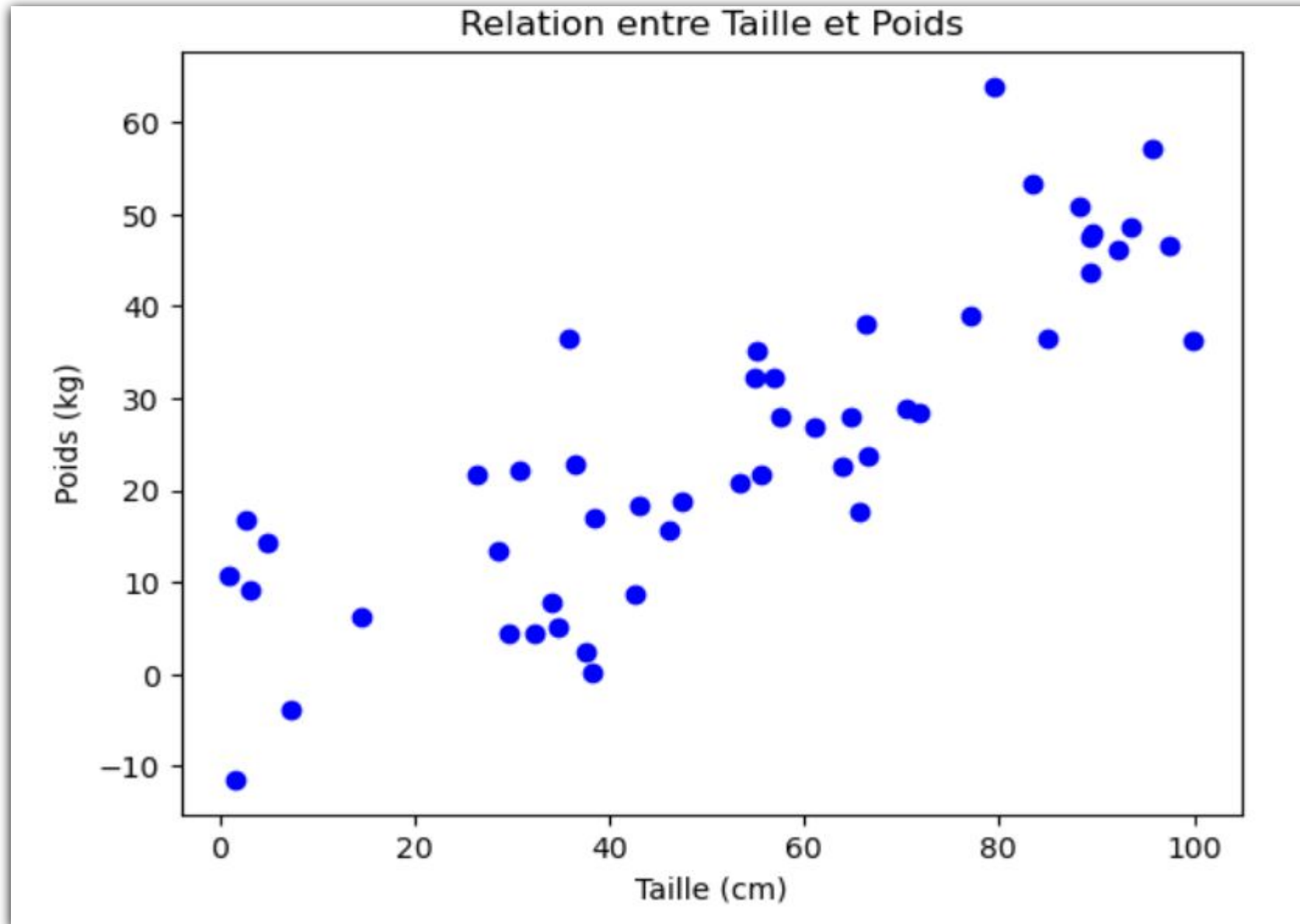
Exemple 1 : Relation taille et poids

```
import matplotlib.pyplot as plt
import numpy as np

# Données
x = np.random.rand(50) * 100 # Taille
y = 0.5 * x + np.random.randn(50) * 10 # Poids (avec une petite variation)

# Création du scatter plot
plt.scatter(x, y, color='blue')
plt.title("Relation entre Taille et Poids")
plt.xlabel("Taille (cm)")
plt.ylabel("Poids (kg)")
plt.show()
```


Exemple 1 : Relation taille et poids



Matplotlib/ Box Plot

- Le box plot est un graphique utilisé pour visualiser la répartition des données, détecter les valeurs aberrantes et comparer plusieurs ensembles de données.
- **Utilisation** : Comparer la distribution des valeurs pour une ou plusieurs variables.
- **Exemple** : Distribution des salaires dans différents départements d'une entreprise.

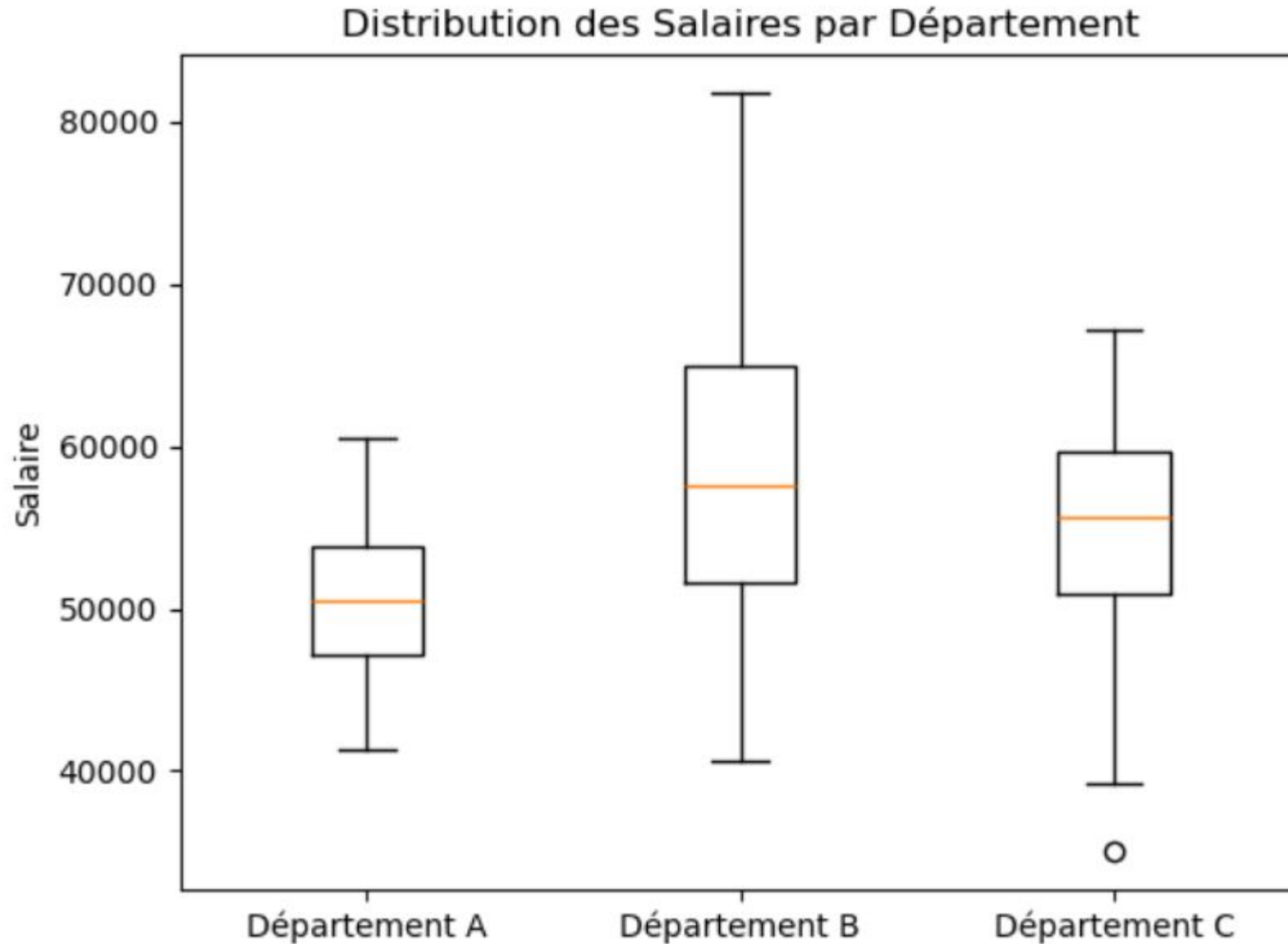
Exemple 2 : Distribution des salaires dans différents départ.

```
import matplotlib.pyplot as plt
import numpy as np

# Données simulées
salaries = [np.random.normal(50000, 5000, 100), # Département A
            np.random.normal(60000, 8000, 100), # Département B
            np.random.normal(55000, 6000, 100)] # Département C

# Création du box plot
plt.boxplot(salaries, labels=["Département A", "Département B", "Département C"])
plt.title("Distribution des Salaires par Département")
plt.ylabel("Salaire")
plt.show()
```

Exemple 2 : Distribution des salaires dans différents départ.



Matplotlib/Histogramme

- Un histogramme permet de visualiser la distribution d'une variable numérique en divisant les données en intervalles (**bins**).
- **Utilisation** : Idéal pour comprendre la répartition des données (par exemple, la fréquence des âges dans une population).
- **Exemple** : Distribution des âges dans un échantillon de personnes.

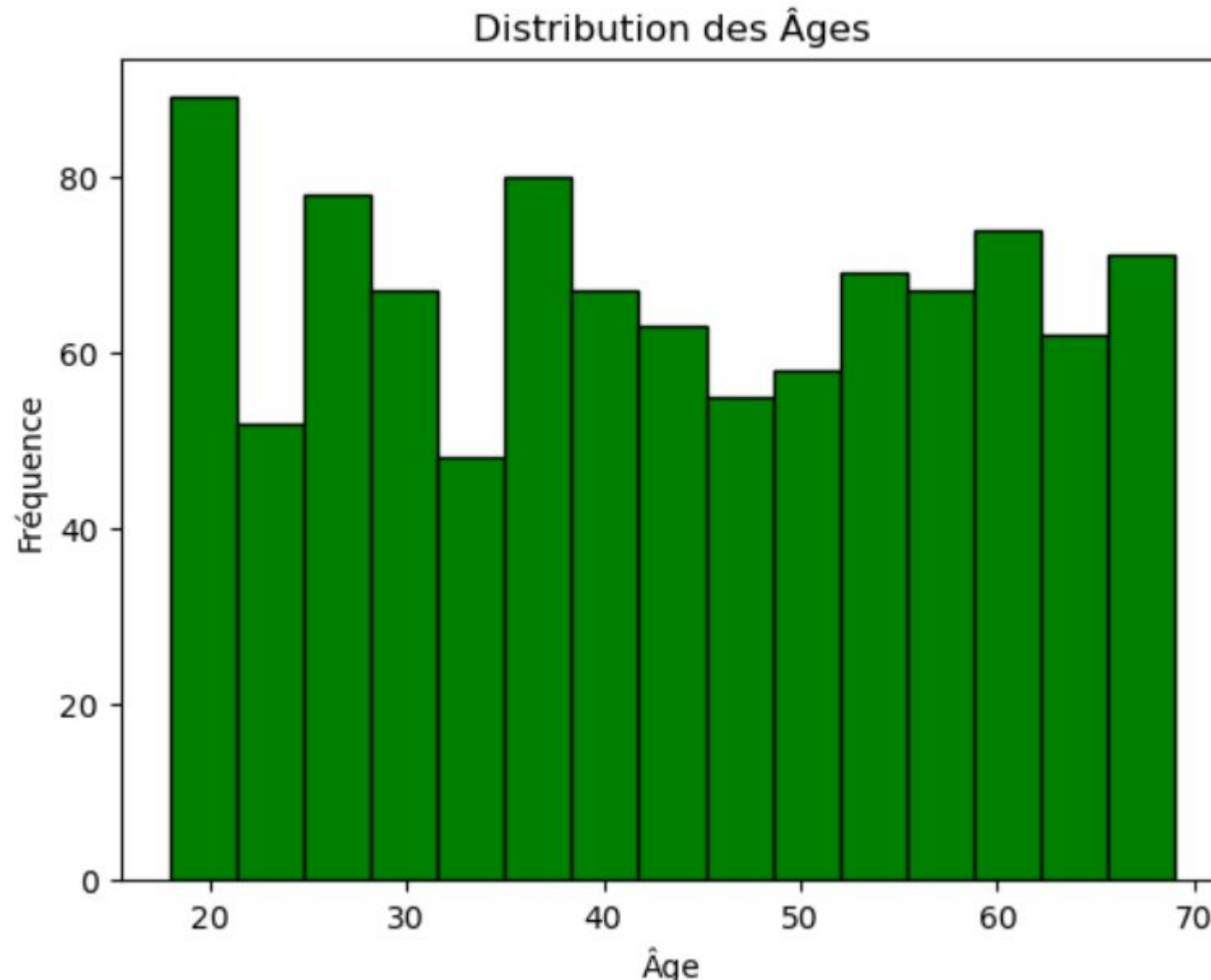
Exemple 3 : Distribution des âges dans un échantillon de personnes.

```
import matplotlib.pyplot as plt
import numpy as np

# Données simulées (âge)
ages = np.random.randint(18, 70, 1000)

# Création de l'histogramme
plt.hist(ages, bins=15, color='green', edgecolor='black')
plt.title("Distribution des Âges")
plt.xlabel("Âge")
plt.ylabel("Fréquence")
plt.show()
```

Exemple 3 : Distribution des âges dans un échantillon de personnes.



Seaborn

- **Seaborn** est une bibliothèque Python basée sur **Matplotlib**, conçue pour simplifier la création de graphiques statistiques.
- Offre des thèmes et des palettes de couleurs préconfigurées, ainsi que des graphiques spécialisés pour l'analyse statistique.
- Explorer les graphiques **Seaborn** couramment utilisés pour la visualisation de données.

Seaborn /Scatter Plot

- Utilisé pour visualiser la relation entre deux variables numériques.
- Possibilité **d'ajouter** une dimension avec des couleurs basées sur une variable catégorielle.

Exemple 1

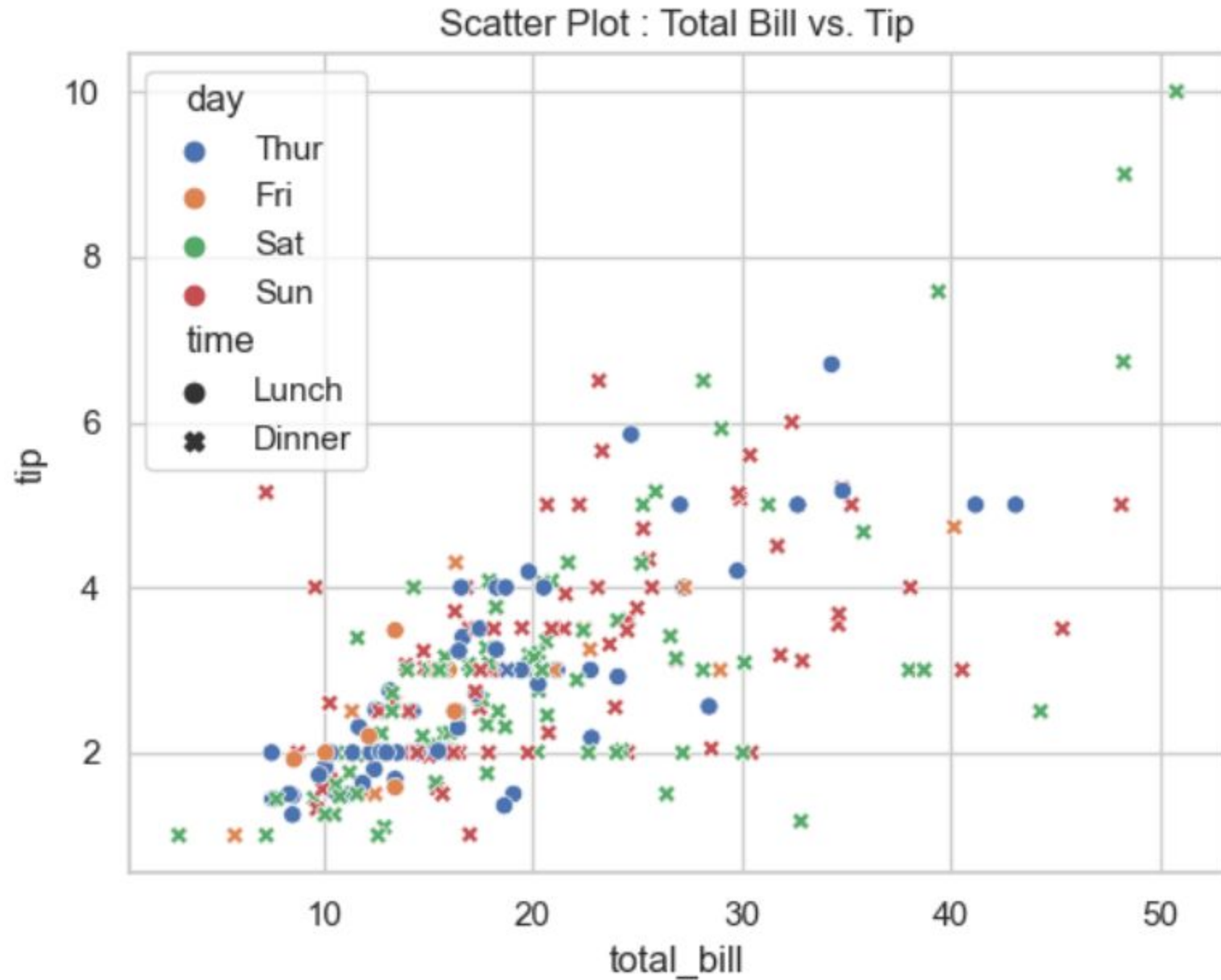
```
import seaborn as sns
import matplotlib.pyplot as plt

# Configuration du thème
sns.set_theme(style="whitegrid")

# Chargement d'un jeu de données intégré
data = sns.load_dataset("tips")

# Création d'un scatter plot
sns.scatterplot(data=data, x="total_bill", y="tip", hue="day", style="time")
plt.title("Scatter Plot : Total Bill vs. Tip")
plt.show()
```

Example 1



Seaborn/Box Plot

- Représente la distribution des données et les valeurs aberrantes.
- Utile pour comparer des groupes sur une même variable.

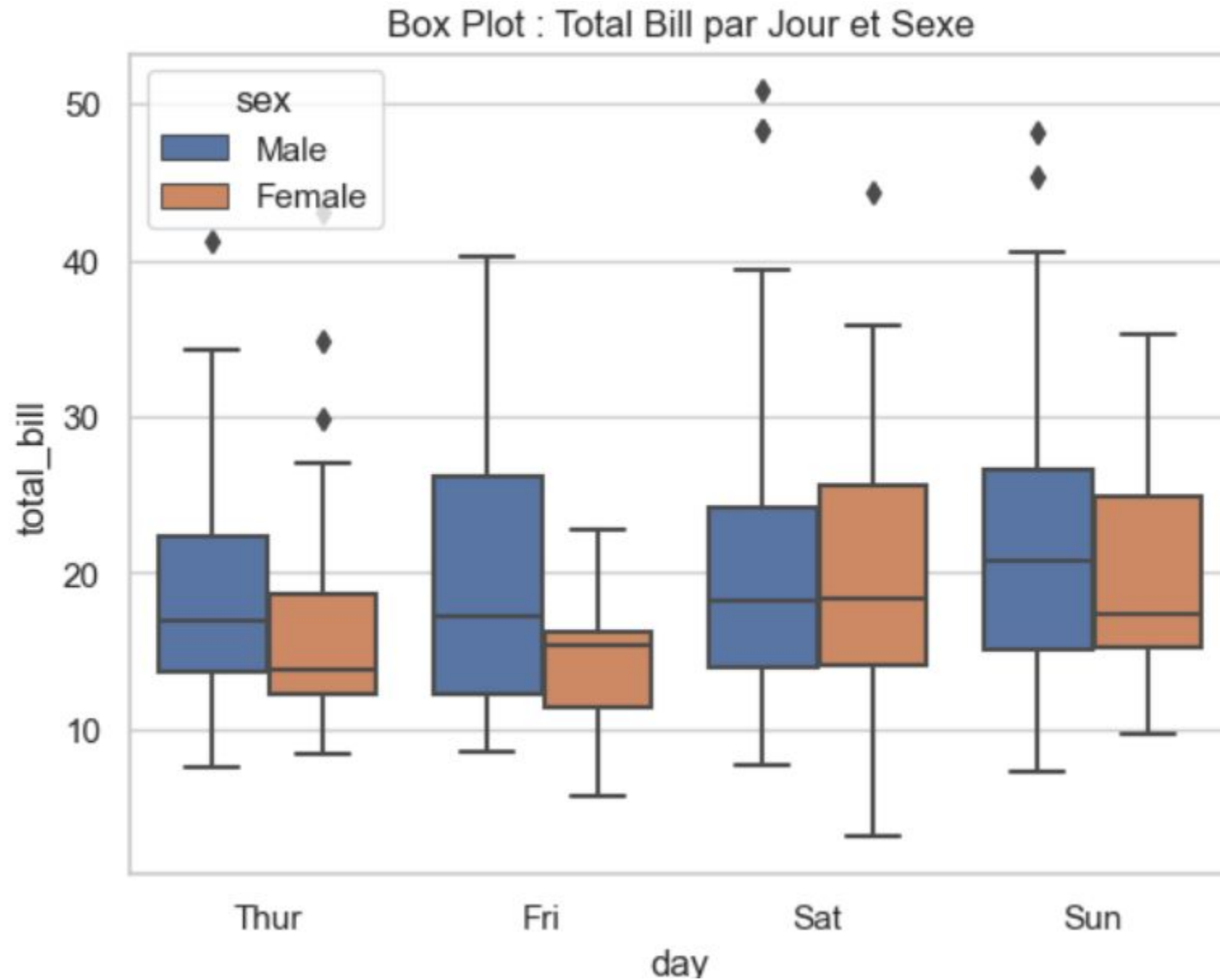
Exemple 2 : Distribution des Factures

```
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
# Suppress FutureWarning from Seaborn related to Pandas groupby observed parameter
warnings.filterwarnings("ignore", category=FutureWarning, module="seaborn")

# Configuration du thème
sns.set_theme(style="whitegrid")

# Chargement d'un jeu de données intégré
data = sns.load_dataset("tips")
sns.boxplot(data=data, x="day", y="total_bill", hue="sex")
plt.title("Box Plot : Total Bill par Jour et Sexe")
plt.show()
```

Exemple 2 : Distribution des Factures



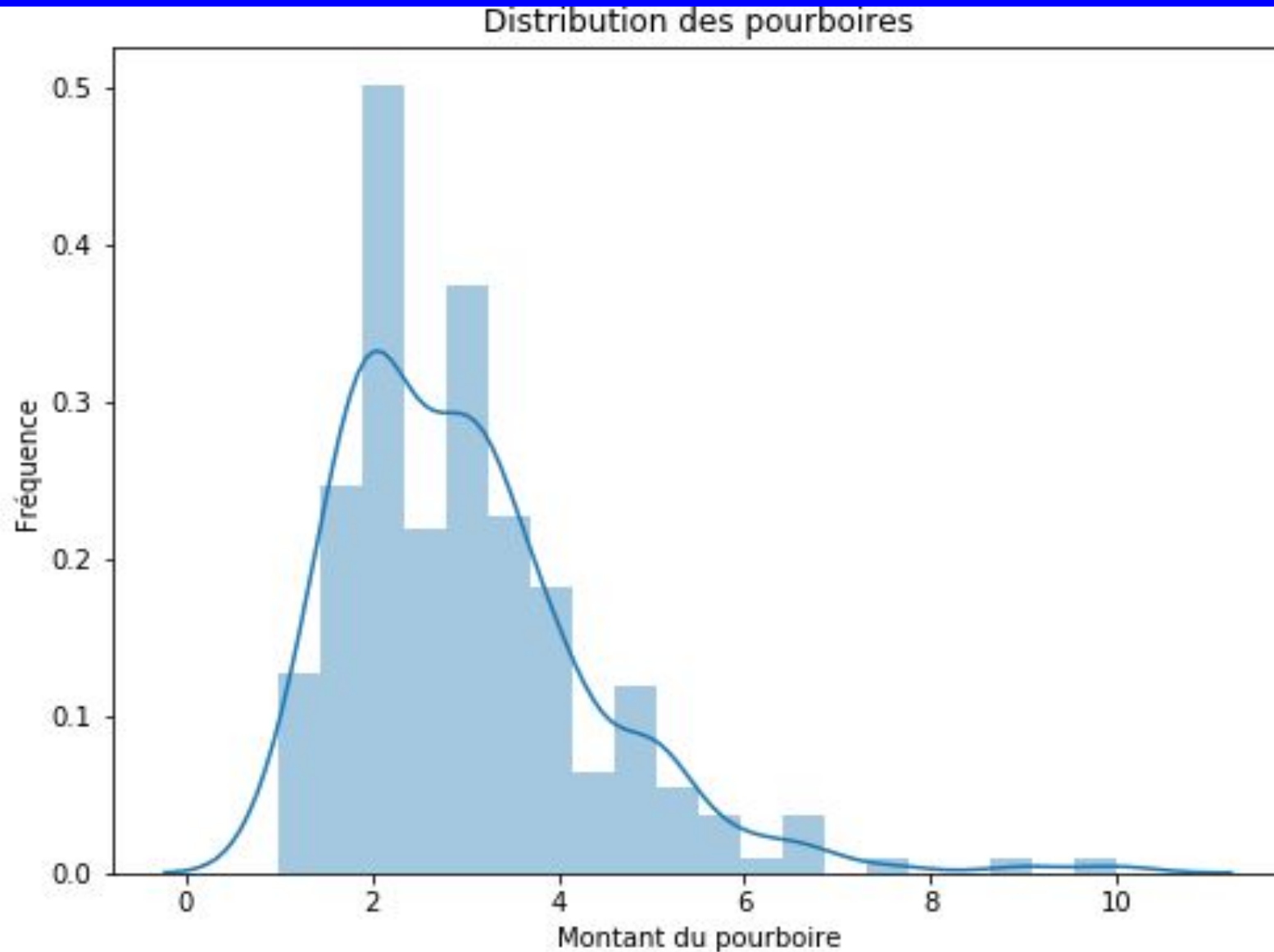
Exemple 3: Distribution des Factures

```
# Import necessary libraries
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset 'tips'
data = sns.load_dataset("tips")

# Create a histogram of the 'tip' variable
plt.figure(figsize=(8, 6))
sns.distplot(data['tip'], kde=True, bins=20) # Note: distplot is now used
plt.title("Distribution des pourboires")
plt.xlabel("Montant du pourboire")
plt.ylabel("Fréquence")
plt.show()
```


Exemple 3 : fréquence des pourboires dans le dataset



Heatmap

- Une heatmap est un graphique utilisé pour afficher des matrices de données numériques sous forme de couleurs.
- Elle permet de visualiser rapidement des corrélations ou des données catégorielles et est particulièrement utile pour explorer des relations complexes dans les données.
- En statistiques, une heatmap est souvent utilisée pour visualiser la matrice de corrélation entre différentes variables.

Example 4

```
# Importing libraries
import seaborn as sns
import matplotlib.pyplot as plt

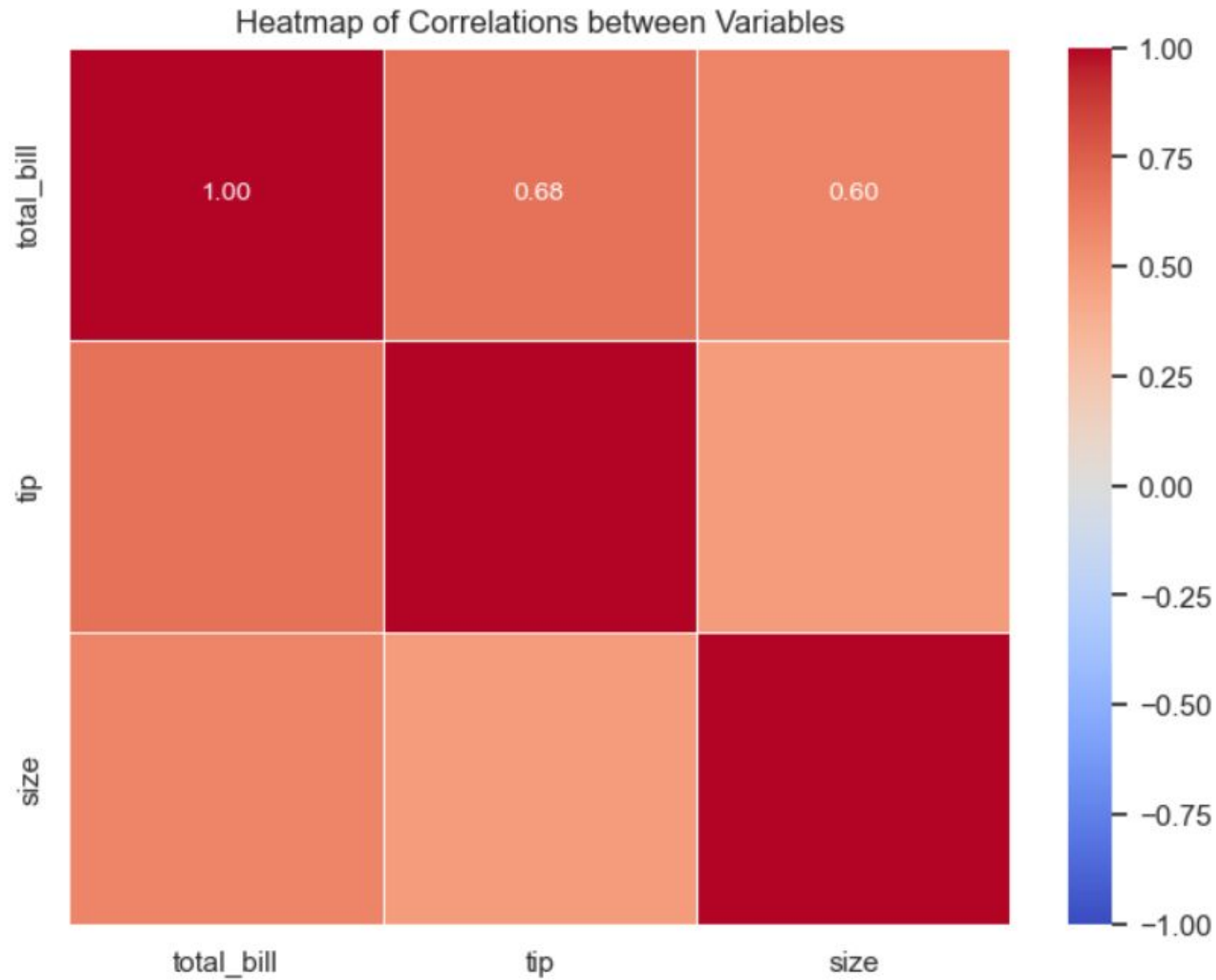
# Load a sample dataset
data = sns.load_dataset("tips")

# Select only numeric columns for correlation calculation
numeric_data = data.select_dtypes(include=['float64', 'int64'])

# Calculate the correlation matrix
correlation_matrix = numeric_data.corr()

# Create the heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", vmin=-1, vmax=1,
            linewidths=0.5, fmt=".2f", annot_kws={"size": 10})
plt.title("Heatmap of Correlations between Variables")
plt.show()
```

Example 4



Graphiques avec Bokeh

- **Définition** : une bibliothèque Python pour la visualisation de données interactives.

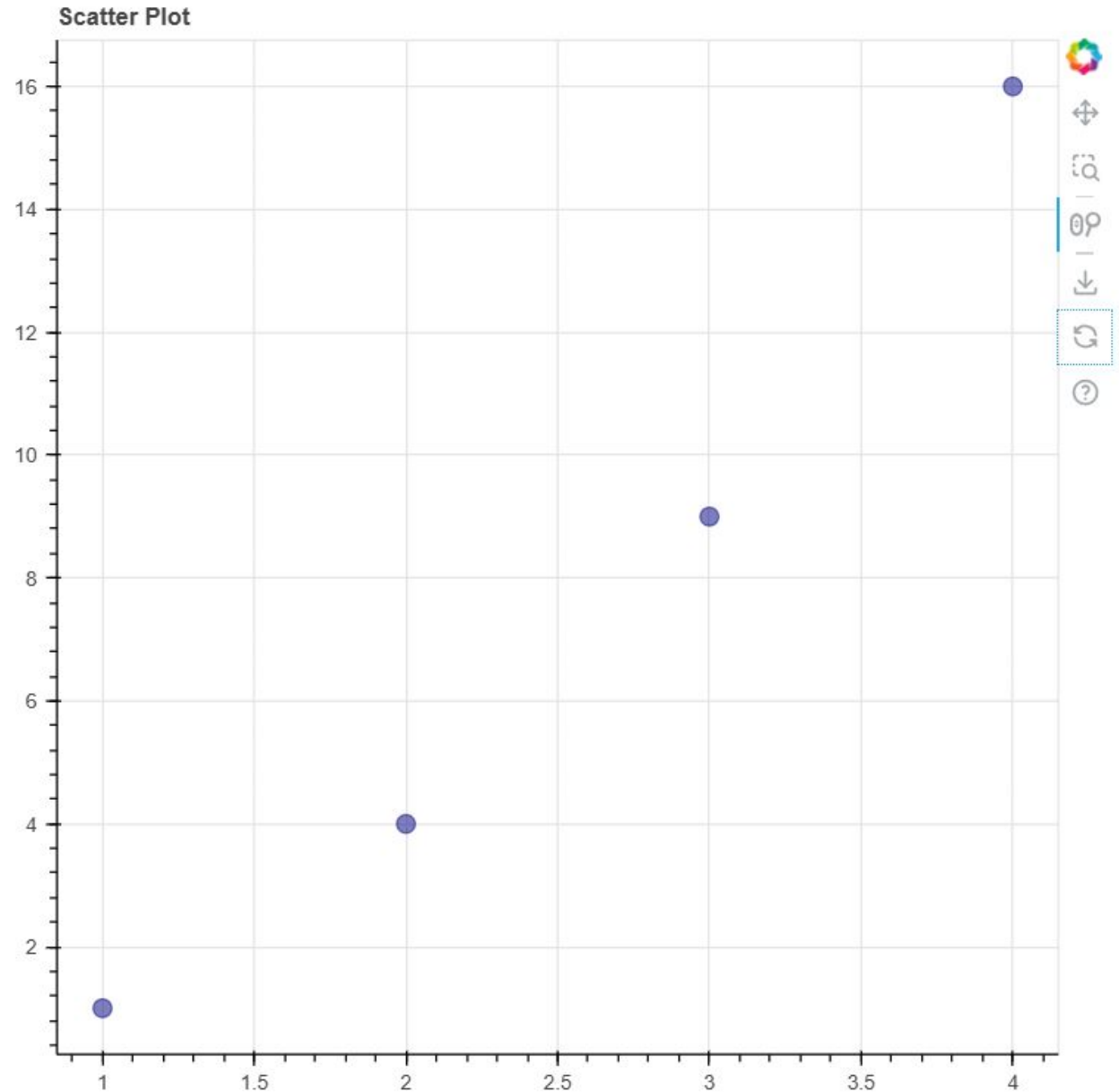
Utilisation : Visualisation de données interactives pour le web

- **Caractéristiques principales** :

- Graphiques interactifs (zooms, survols, filtres)
- Très adapté pour des visualisations en temps réel
- Intégration facile dans des applications web via HTML et JavaScript
- Prise en charge des graphiques complexes, tels que les cartes et les graphiques multidimensionnels

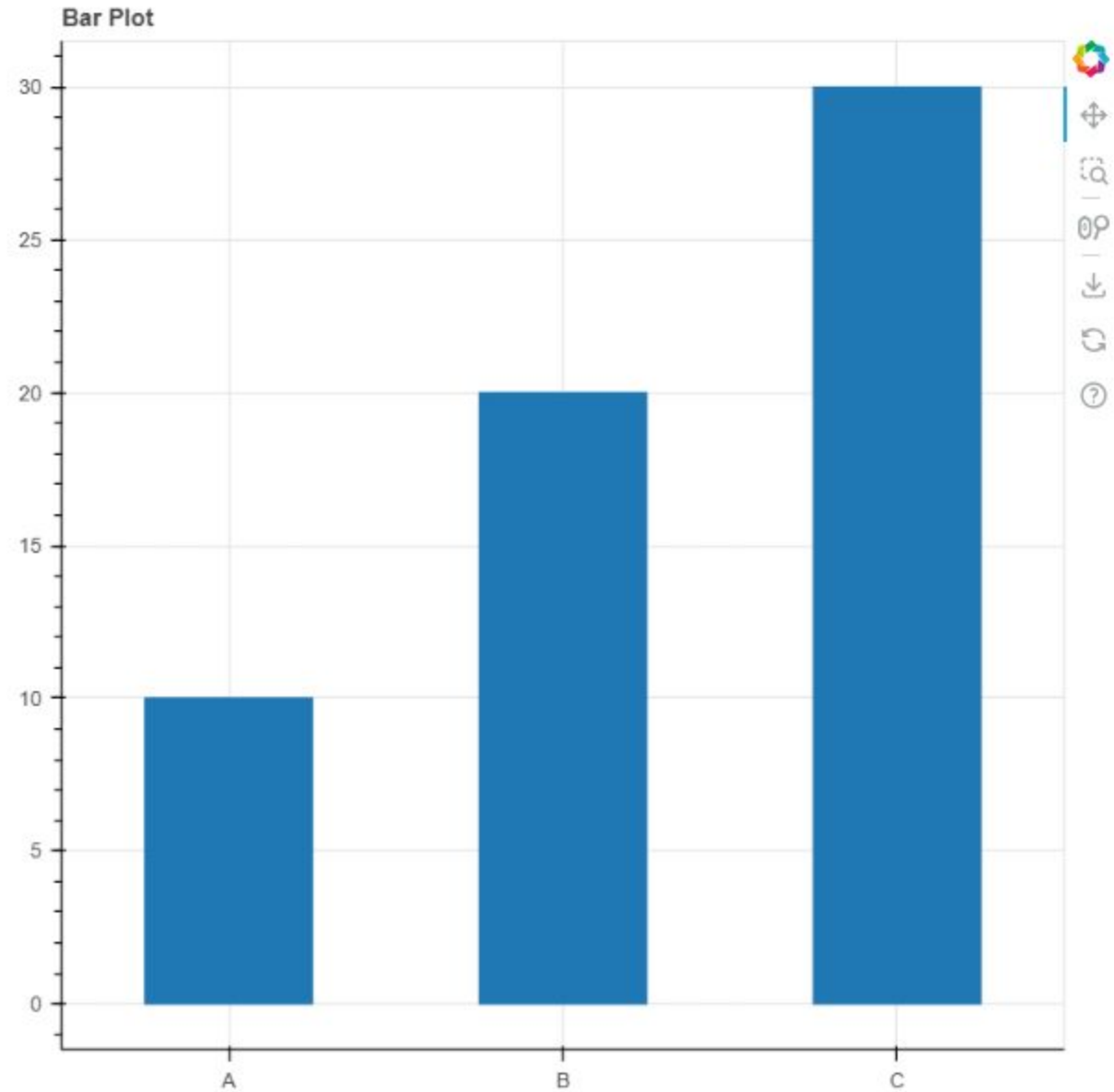
Scatter Plot

```
from bokeh.plotting import figure, show
p = figure(title="Scatter Plot")
p.scatter([1, 2, 3, 4], [1, 4, 9, 16], size=10, color="navy", alpha=0.5)
show(p)
```



Bar Plot

```
from bokeh.transform import dodge
categories = ["A", "B", "C"]
counts = [10, 20, 30]
p = figure(x_range=categories, title="Bar Plot")
p.vbar(x=categories, top=counts, width=0.5)
show(p)
```



Histogramme

```
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.layouts import column

# Générer des données aléatoires
data = np.random.normal(0, 0.5, 1000)

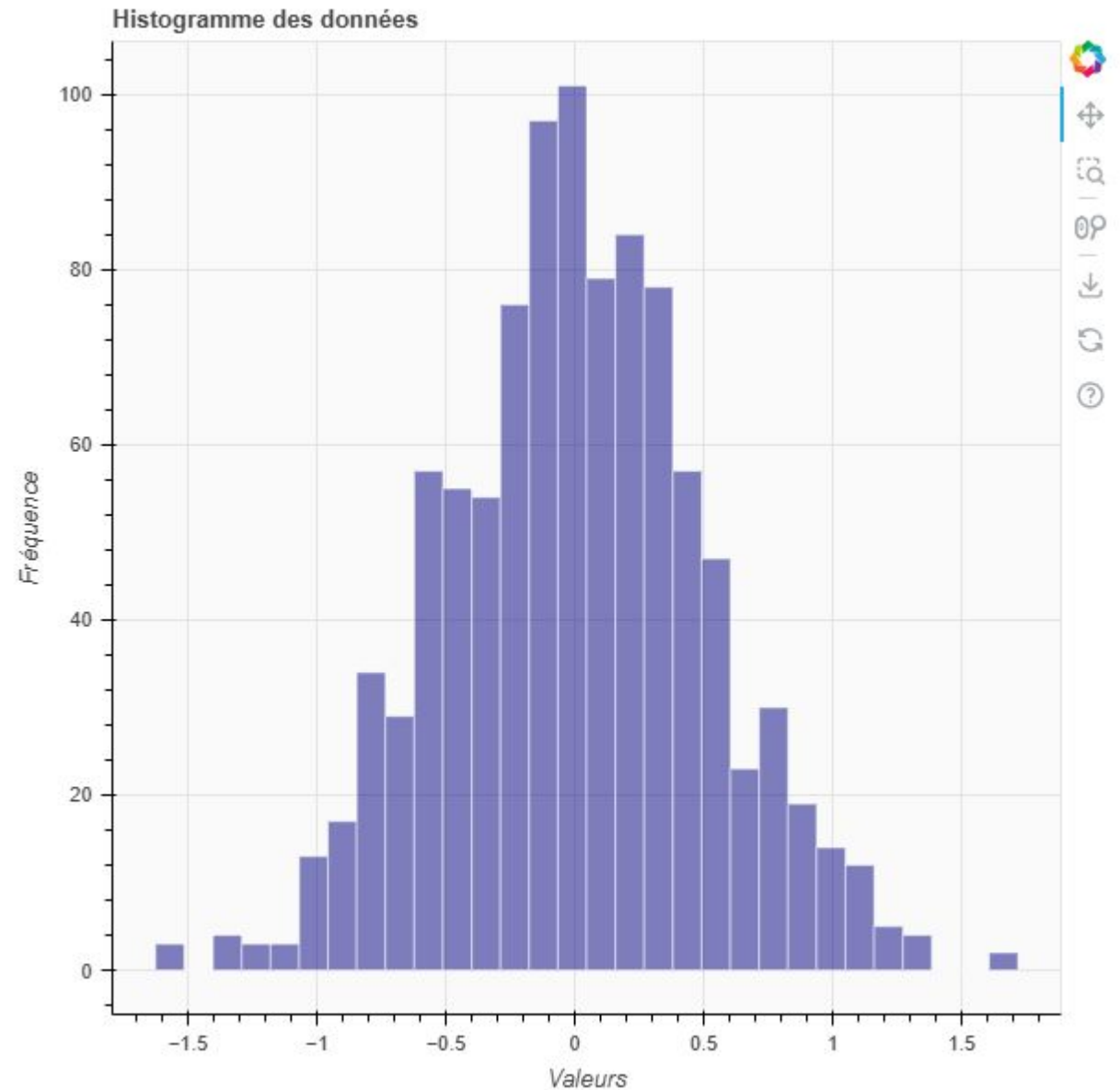
# Calculer l'histogramme
hist, edges = np.histogram(data, bins=30)

# Créer la figure Bokeh
p = figure(title="Histogramme des données",
           x_axis_label="Valeurs",
           y_axis_label="Fréquence",
           background_fill_color="#fafafa")

# Ajouter l'histogramme à la figure
p.quad(top=hist, bottom=0, left=edges[:-1], right=edges[1:],
       fill_color="navy", line_color="white", alpha=0.5)

# Afficher l'histogramme dans un notebook Jupyter (facultatif)
output_notebook()

# Afficher la figure
show(p)
```



Conclusion

- La visualisation de données est un pilier de l'analyse exploratoire et de la communication des résultats.
- Les trois bibliothèques **Matplotlib**, **Seaborn**, et **Bokeh** sont des outils complémentaires qui offrent diverses solutions pour la visualisation de données.
- Selon le besoin (interactivité, exploration de données, personnalisation des graphiques), l'une de ces bibliothèques sera plus adaptée.
- La maîtrise de ces bibliothèques vous permettra de mieux analyser, comprendre, et communiquer vos données de manière efficace.